

Laya

Architecture, How It Works, and Where It Wins

The open-weights System 1 decision engine:
typed decisions in a single forward pass.

Companion explainer for the intent-router-laya project.
Facts verified against the official Laya repository (v0.3.24)
and our live run of laya-typed-decisions on 2026-10-02.

1. What Laya is — the 30-second version

Laya is an **open-weights "System 1" decision engine**. You give it a piece of text plus a set of typed questions — "which department?", "how urgent on a 1–5 scale?", "is this a jailbreak attempt?" — and it answers all of them in a **single neural-network forward pass**, returning structured answers with calibrated probabilities. No text generation, no prompt engineering, no API key, no per-call bill. Apache-2.0 licensed; the weights are yours to self-host and fine-tune.

The name is deliberate: in dual-process psychology, System 1 is fast, automatic judgment — exactly the slot Laya fills in an agent stack, with a slower LLM playing System 2 reviewer for the hard cases.

2. The core idea: decisions, not generations

A general LLM answers a classification question by generating text ("The intent is billing_inquiry"), one token at a time. That is expensive (seconds, cents) and unreliable (you must parse the text, and the format can drift).

Laya treats a decision as a **first-class output type**. The question declares its answer shape up front, and the model emits exactly that shape:

Question type	You declare	You get back
choice	instructions + named options with descriptions	winning label, probability per option, confidence
noul	instructions (+ optional false/true descriptions)	calibrated P(true) in [0, 1]
score	instructions + ordered rubric, e.g. ["safe", "low", "medium", "high", "critical"]	expected rubric level, full distribution, confidence

Because the output vocabulary is fixed by the question, there is nothing to parse and nothing that can hallucinate a format. The model never generates a single token of free text — hence **non-autoregressive**: latency is one forward pass, ~33 ms on a T4 GPU for a typical call, instead of hundreds of milliseconds to seconds for an autoregressive judge.

3. Architecture deep dive

3.1 Encoder backbone

Each Laya checkpoint is a fine-tuned **bidirectional encoder** (the BERT family), not a generative LLM:

- laya (English) — built on ModernBERT-large, ~421M parameters.
- laya-multilingual — ~322M parameters, 100+ languages, up to 8,192 tokens of context for long documents.
- laya-typed-decisions — the English checkpoint further fine-tuned for typed decisions; the one our router uses.

Bidirectional encoders read the whole input at once (no left-to-right decoding), which is why one pass suffices. At 322–421M parameters the model fits comfortably on a single GPU, an Apple Silicon laptop, or even a CPU server — a different universe from the multi-GB generative models it replaces for decision work.

3.2 The Router: right checkpoint per request

laya.Router() inspects each input (script, language signals) and dispatches it to the

best checkpoint — English text to the English model, Hindi/Spanish/ etc. to the multilingual one. You can also pin a checkpoint per call. The point: **one API, many languages, no per-language models to maintain.**

3.3 The decision heads

For each question, the model encodes state + question and a small task-specific head converts the encoding into the declared answer type:

- **Choice head.** Each option's description is embedded alongside the state; the head scores every option and applies softmax. Output: the winning label plus a full probability distribution over your options. Because options are described in natural language ("billing: invoices, payments, refunds"), adding a new intent means adding a sentence — **no retraining.**
- **Noul head.** Scores two semantic slots, false vs. true, and returns the calibrated probability of true. This is the binary-question workhorse: "is this phishing?", "does the user threaten to cancel?", "should a human review this?"
- **Score head.** Treats your rubric as an ordered scale and returns the **expected level** (e.g. 2.04 on a 0–4 safe→critical scale) plus the distribution across levels. Ordinal, not categorical: "medium" is between "low" and "high", and the head knows it.

3.4 One forward pass, many questions

All questions in a `predict(state, questions)` call are answered **in parallel in a single forward pass** — they become rows in one batch, not separate model calls. Our router exploits this: intent (choice) + human-review (noul) + utterance-type (choice) + guardrail-risk (score) are decided together, which is why batching 10 questions costs ~72 ms instead of 10× the single-question cost on a T4.

3.5 Calibration: trusting the numbers

Raw neural-network probabilities are usually overconfident. Laya invests heavily in making its numbers mean what they say:

- **Temperature scaling** — a per-bucket temperature fitted on labeled data rescales logits so "0.8 confidence" is right ~80% of the time. (Our live run surfaced the honest warning: the shipped checkpoint flags some temperatures as uncalibrated — temperatures must be refit on your data.)
- **Abstention thresholds** — per-option-count cutoffs (`fit_abstention_thresholds`) plus an opt-in `min_confidence` gate: below the bar, the model abstains instead of guessing.
- **Histogram binning** — a fallback recalibration for buckets temperature scaling can't fix.
- **Evaluation harness** (`laya.eval`) — accuracy, ECE, Brier score, selective

accuracy gated in CI, so quality changes are reviewable diffs.

Measured result: expected calibration error (ECE) of **0.081** after temperature fitting — roughly 3× better than the closed alternative's 0.246.

3.6 Training: RLCD

Laya's checkpoints are trained with **reinforcement learning against strictly proper scoring rules (RLCD)** — the model is rewarded for assigning high probability to correct answers and for being honest about uncertainty (a proper scoring rule, by definition, is maximized only by reporting your true beliefs). This is why the probabilities are meant to be taken literally, and why the project ships a fine-tuning notebook/script (Kaggle 2×T4 or Apple Silicon): on the project's 2,000-decision benchmark, the fine-tuned laya-typed-decisions scores **0.766 accuracy** vs **0.362** for the base checkpoint on the same decisions. Fine-tuning is where the accuracy lives.

4. How Laya differs from the alternatives

	Laya	LLM-as-judge	Classic classifier	TypeSafe Jev	OpenAI Decisions API
Output	Typed, structured, guaranteed	Free text you must parse	Fixed label set	Typed (choice/noul)	Typed decisions
Latency	~33 ms (T4); one forward pass	Seconds (autoregressive)	~ms	~236–276 ms p50	Unknown (limited preview)
Cost	\$0 self-hosted	Per-token API billing	\$0 self-hosted	\$0.042 / 1M tokens	Unknown
Weights	Open, Apache-2.0	Closed	Open	Closed API	Closed
Fine-tunable	Yes (RLCD notebook/script)	No	Yes	No	No
Zero-shot new labels	Yes — describe the option	Yes — prompt it	No — retrain	Yes	Yes
Multilingual	100+ languages, one model	Yes	One model per language	Not benchmarked	Unknown
Calibration tooling	Temperatures, abstention, ECE gates	None built-in	Manual	Weaker (ECE 0.246)	Unknown
>20 options	Weak (0.425 on Banking77)	Fine	Fine	Strong (0.870)	Unknown

The one-line summary: **Laya is what you get when you stop using a text**

generator to make decisions and train a small model to output decisions directly — keeping the zero-shot flexibility of prompting (options are just descriptions) while recovering the speed, cost, and determinism of a classifier.

5. Best use cases

Intent / ticket routing (our use case). One call classifies the intent, estimates whether a human should look, and scores risk — the exact triple our router's policy gates on. New intents ship as config, not retraining.

Guardrails and safety scoring. score over a severity rubric and noul for "is this a jailbreak / phishing / PII leak" give you a cheap, always-on safety layer in front of (or beside) the LLM. Our router hard- blocks at guardrail ≥ 3.0 without ever waking System 2.

Agentic tool/workflow selection. An agent deciding which tool to call, which branch of a workflow to take, or whether to ask a clarifying question needs a 30 ms judgment, not a 3 s essay.

Human-in-the-loop triage. noul ("should a human review?") plus the min_confidence abstention gate routes only the uncertain cases to people — the economical shape of every real-world automation.

Evaluation harnesses. Replacing an LLM judge with typed decisions makes evals deterministic, 10–100× cheaper, and committable to CI (laya.eval gates accuracy/ECE on every PR upstream).

Multilingual classification. One 322M model covering 100+ languages replaces a rack of per-language classifiers.

6. Honest limitations (including what our live run found)

- **Zero-shot is a starting point, not the product.** Base-checkpoint accuracy (0.362) roughly doubles after fine-tuning (0.766). Budget for the RLCD loop or our encoder track before production.

- **Calibration must be refit on your data.** The shipped checkpoint itself warns that some temperatures are uncalibrated; we observed confidences of 0.016–0.235 even on correct answers. Tune gate thresholds empirically.

- **Many options degrade.** Past ~20 options in one question, accuracy falls (0.425 on 77-label Banking77) because options share a fixed token budget. Mitigation: predict_shortlist narrows to top-k first, then decides.

- **Position bias is real.** Where an option sits in the list shifts its probability; the option_order rotation trick averages it out.

- **CPU latency is seconds, not milliseconds.** We measured 4–11 s per

decision on CPU; the 25–45 ms figure needs a GPU (or the ONNX/TileLang fast paths).

- **A score head is not a safety certification.** Treat the guardrail as a learned heuristic with measured error rates, not a guarantee.

7. Glossary

- **System 1 / System 2** — fast intuitive judgment vs. slow deliberate reasoning (Kahneman). Laya is built to be the System 1 in an agent stack.
- **Non-autoregressive** — the model emits its answer in one pass instead of generating tokens left-to-right.
- **Typed decision** — a question whose answer shape (choice/noul/score) is declared up front and guaranteed in the output.
- **Noul** — Laya's name for a calibrated binary (false/true) judgment, returned as P(true).
- **RLCD** — reinforcement learning from calibrated decisions: training against strictly proper scoring rules so probabilities stay honest.
- **Temperature scaling** — dividing logits by a fitted constant to align confidence with empirical accuracy.
- **ECE** — expected calibration error; lower means "0.8 confident" is right closer to 80% of the time.

Sources: Laya GitHub README and BENCHMARKS.md (v0.3.24, Apache-2.0); our live verification notes in docs/SDR.md. Benchmark figures are the project's own T4 measurements; Jev figures are third-party published, not measured by the Laya project.